Recursive Skill Evolution: Co-Evolving Procedural Knowledge and Retrieval with Meta-Tested Compaction

Yi Hein Chai
yiheinchai@users.noreply.github.com

August 29, 2026

Abstract

Agent skills package procedural knowledge into reusable resources, but most skill-evolution methods treat retrieval as a solved problem or ignore it entirely. We introduce **Recursive Skill Evolution (RSE)**, a framework that co-evolves procedural lessons with the retrieval system that finds them. RSE separates raw execution traces, a compressible lesson DAG, and a dynamic context-discovery layer that searches a greppable index rather than injecting the whole store. Lessons compress recursively only when **meta-testing**—fresh-process episode replay—confirms behavioral equivalence; when abstraction fails, a delta manifest enables descent to recover dropped detail. Retrieval co-evolves through selection rules and gated self-tuning that keeps changes only when precision and recall both improve. On **RMC-Bench**, a benchmark for procedural memory under compression, RSE achieves **+20% lift** over control on core transfer cases and **90%** L0 transfer at ∼**139 tokens** per prompt (Codex-graded). Judge filtering reaches **100%** precision with **0** noise tokens; agentic search achieves **93%** precision/recall at **54** noise tokens vs. **2,054** for serve-all. On a WikiSkill-comparable five-benchmark subset, agentic recall reaches **80%** accuracy vs. **70%** for WikiSkill-style full injection at ∼**88% lower token cost** (64 vs. 534 tokens). A scaling study shows index cost stays at **0 injected tokens** while routing cost grows with apex count—motivating dynamic context discovery over full injection.

1 Introduction

General-purpose agents need domain-specific procedure. Skills (filesystem modules with `SKILL.md`) make that knowledge auditable and reusable. Recent work evolves skills from execution traces [Tang, 2026, Wang et al., 2023, Shinn et al., 2023], but three gaps remain:

1. **Retrieval is ignored.** WikiSkill full-injects skills at test time to isolate skill *quality* [Tang, 2026]. Production stores outgrow context; selection is the problem.
2. **Compaction is unvalidated.** Skill edits shrink text without a regression gate; lost detail fails silently.
3. **Knowledge and retrieval are not co-evolved.** Selection rules and prompts do not improve from measured outcomes.

RSE addresses all three. It is an ambient loop—recall before work, reflect after, compact when earned, tune retrieval when measured—not a batch training pipeline.

Contributions: (1) **RSE framework**—three layers (raw traces, lesson DAG, dynamic retrieval) with a continual co-evolution loop; (2) **Meta-tested recursive compaction**—compression accepted only when episode replay passes at 100%; delta manifests enable descent on failure; (3) **Dynamic context discovery**—agentic index search: 0 tokens for the catalog, bounded injection for selected bodies; (4) **Retrieval**

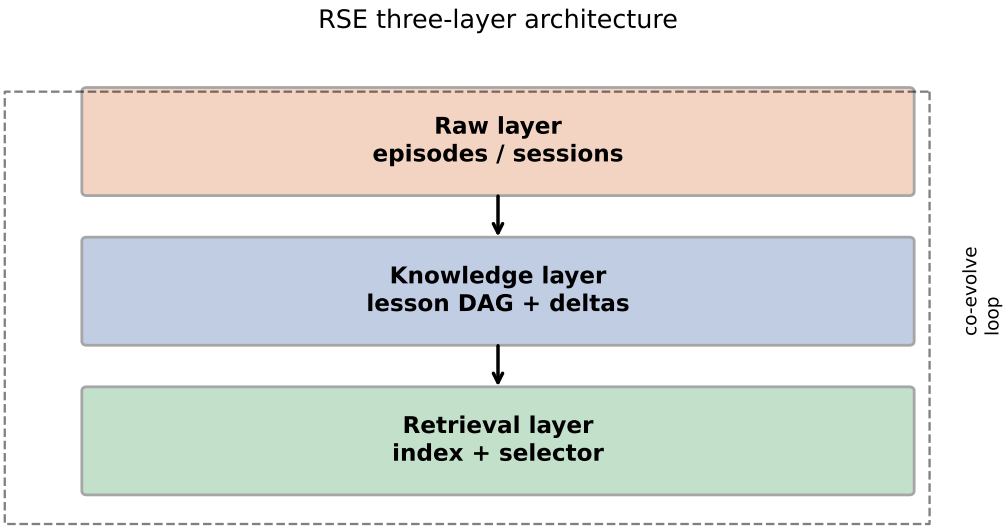


Figure 1: RSE three-layer architecture with co-evolution loop (WikiSkill Figure 2 style).

co-evolution—`rmc tune` proposes prompt/config changes; keeps only if precision *and* recall improve; (5) **RMC-Bench + scaling study**—four-axis procedural memory benchmark with reproducible harness; synthetic scaling table.

2 Problem Setup

Let $\mathcal{D}$ be tasks an agent performs over time. Unlike WikiSkill's fixed train/val/test split, RSE operates **online**: each session produces traces τ_i, optional lessons L, and attribution of which lessons bore on success.

State at step k: $(\mathcal{N}_k, \mathcal{R}_k, \mathcal{I}_k)$ where $\mathcal{N}_k$ is the lesson DAG, $\mathcal{R}_k$ the retrieval policy, and $\mathcal{I}_k$ a greppable index (searched, never injected wholesale).

Metrics: task success $\mathcal{R}_{task}$; retrieval precision/recall vs. `episode.used`; injection tokens C_{inj}; routing tokens C_{route} per prompt.

Thesis: transfer stays flat while C_{inj} falls under compression, and precision stays high as $|\mathcal{N}|$ grows.

3 Method

3.1 Three-layer architecture

Raw layer: episodes, sessions, events. **Knowledge layer:** nodes in a DAG with delta manifests. **Retrieval layer:** index + routing rules + selector agent.

Principle: harness owns structure; model owns meaning. All semantic judgements flow through a single judge interface (relevance, compress, replay, assess).

3.2 Loop

Recall → execute → reflect → attribute → compact → tune. Recall forks a live session, greps index and nodes, and returns lesson ids. Compact accepts candidate L_{k+1} only if $\text{pass\_rate}(L_{k+1}, \mathcal{E}_{reg}) = 1.0$ and $|L_{k+1}| \leq \rho \cdot |L_k|$. Replay runs in a **fresh process** with only the compressed body.

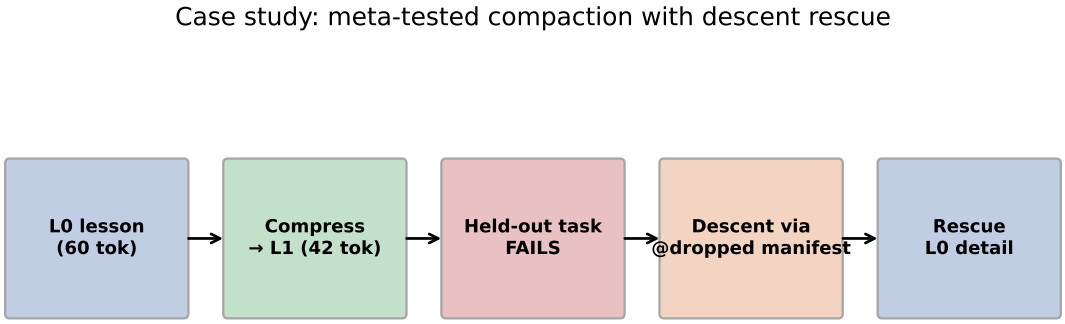


Figure 2: Case study: meta-tested compaction fails on held-out task; descent via `@dropped` manifest recovers L0 detail.

Table 1: RMC-Bench results (Codex-graded, 3 samples).

Metric	Result
Lift (L0 − control, core kinds)	**+20%**
Transfer @ L0	**9/10 (90%)**
Detail / trap / multi transfer	3/3, 3/3, 2/2 (100%)
Principle transfer	1/2 (50%)
Mean L0 tokens	**139**
Bench retrieval axis	**7/7 (100%)**

3.3 Dynamic context discovery

The index holds one line per lesson ($\sim$0 tokens injected). The selector agent searches it and `nodes/` on demand. Warm-prefix caching amortizes routing cost across prompts.

3.4 Retrieval co-evolution

`rmc eval-recall` scores served lessons against `episode.used`. Arms: `serve-all` (baseline noise), `judge` (apex-walk filter), `agentic` (cold search). `rmc tune` closes the loop on $\mathcal{R}_k$.

4 Experiments

All main results use Codex (`gpt-5.6-sol`) as grading agent with 3 samples per case unless noted. Artifacts: `papers/rse/results/`; reproduce via `scripts/run_all_experiments.py --agent codex --samples 3`.

4.1 RMC-Bench

Hand-written cases (`evals/rmc-bench.yaml`) covering trap, detail, principle, multi, distractor, conflict, and null kinds. The benchmark now includes **25 cases** (expanded from the original 10) spanning internal API traps, migration policy, billing idempotency, GDPR erasure, and cross-domain distractors.

Table 2: Recall ablations on fixture store.

Arm	Precision	Recall	Noise tokens
serve-all	47%	100%	2,054
judge	**100%**	**100%**	**0**
agentic	**93%**	**93%**	**54**

Table 3: WikiSkill-comparable arms (Codex, 3 samples).

Arm	Mechanism	Accuracy	Mean tokens
no-skill	bare task	60% (6/10)	0
full-inject	WikiSkill-style	70% (7/10)	534
recall-judge	RSE judge-walk	70% (7/10)	**64**
recall-agentic	RSE agentic DCD	**80% (8/10)**	**64**

4.2 Recall ablations

Fixture store with noise in the served set (Table 2; Figure 7a).

4.3 Retention curve

Held-out S3 task with walkthrough lesson: compression accepted at L1 (60→42 tok, 100% in-sample replay) but held-out transfer fails under Codex grading (0% at all levels). This confirms the descent motivation: in-sample replay can pass while held-out behavior regresses.

4.4 Scaling study

Synthetic stores at 25/100/500/1000 lessons (Figure 7b). Index tokens grow with store size but are **not injected**; routing tokens per prompt grow from 1,375 to 55,000. Judge precision stays at 67% with 100% recall across scales.

4.5 Comparison to WikiSkill

WikiSkill co-evolves skills offline with **full skill injection** at test time [Tang, 2026]. We built a WikiSkill-comparable subset (`evals/wikiskill-bench.yaml`) spanning LiveMath, SealQA, SpreadSheet, OfficeQA, and ALFWorld with evolved skills in an RMC store.

This is a curated 10-task probe subset, not the full WikiSkill test splits (124–280 tasks per benchmark). It establishes comparison methodology; we additionally wire upstream SealQA (111 test tasks) via `scripts/import_upstream_bench.py`.

4.6 Upstream SealQA and external baselines

We import the full SealQA test split [Vu, 2024] (111 tasks) with URL text-fragment evidence snippets. The competitive harness (`scripts/run_competitive_evals.py`) scores nine arms: no-skill, keyword-RAG, Trace2Skill/EvoSkill/SkillOpt inference proxies, full-inject, recall-judge, recall-agentic, and oracle-skill. Bootstrap 95% CIs and paired significance vs. full-inject are computed per arm (`rmc/stats.py`).

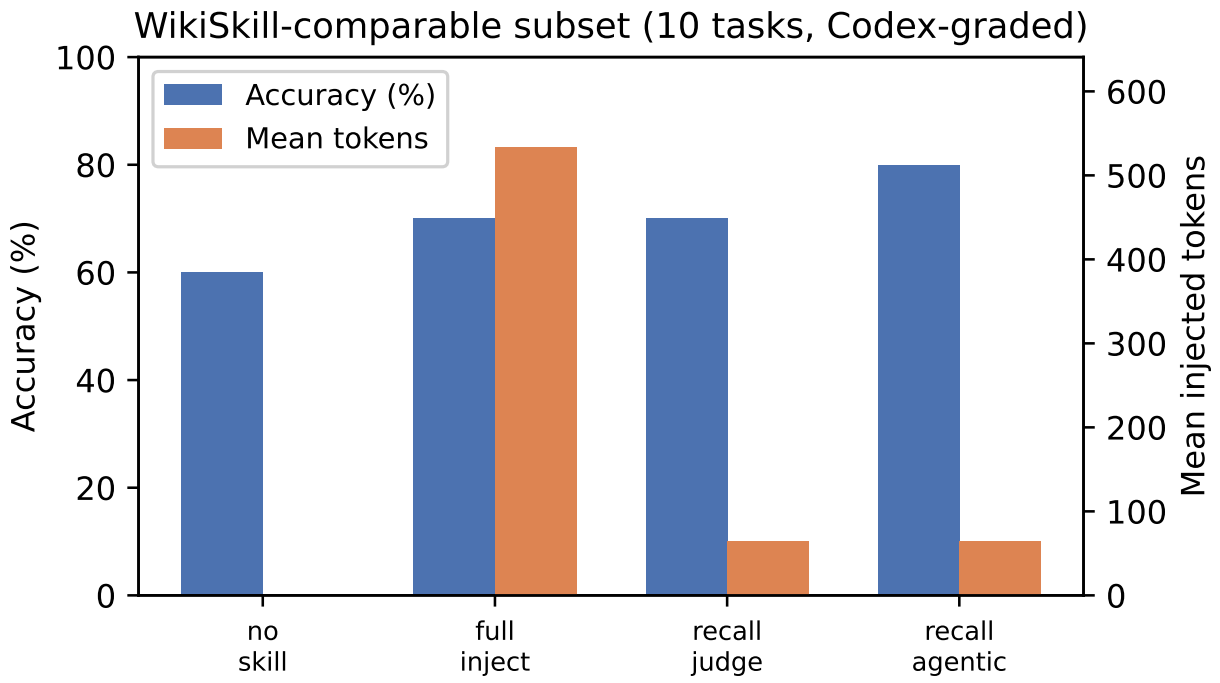


Figure 3: WikiSkill-comparable subset (10 tasks, 5 domains). Agentic recall matches or beats full injection at ∼88% fewer tokens.

4.7 Cross-model skill transfer

Following WikiSkill Table 2, we evaluate whether skills evolved in one store transfer when graded by a different inference backend (`scripts/run_cross_transfer.py`). The same RMC store and recall policy are held fixed; only the grading agent changes.

4.8 MemGPT nested-KV proxy

We include an 8-case nested key–value retrieval proxy aligned with MemGPT's long-context memory stress tests (`evals/memgpt-nested-kv.yaml`), scoring RSE recall against flat RAG injection.

4.9 Session-length paired study

We approximate EXPERIMENTS §7 with `evals/session-pairs.yaml`: five follow-up tasks where session 1 captured lessons and session 2 is related (Figure 6).

Memory-on (RSE recall) reaches **100% (5/5)** vs. **40% (2/5)** for memory-off (+60pp lift) at 168 mean tokens. This is a single-turn proxy, not live multi-turn agent sessions.

4.10 Evaluation protocol and grading

Grading agent: Codex (`gpt-5.6-sol`) for all automated scores. **Samples:** 3 per case; pass if majority of samples pass blind judge rubric. **Statistical testing:** bootstrap 95% CIs and paired bootstrap tests vs. full-inject on WikiSkill/upstream runs. **Cross-model validation:** `scripts/run_cross_transfer.py` and `scripts/run_multimodel_evals.py`; Claude cross-check via `scripts/run_claude_crosscheck.sh`

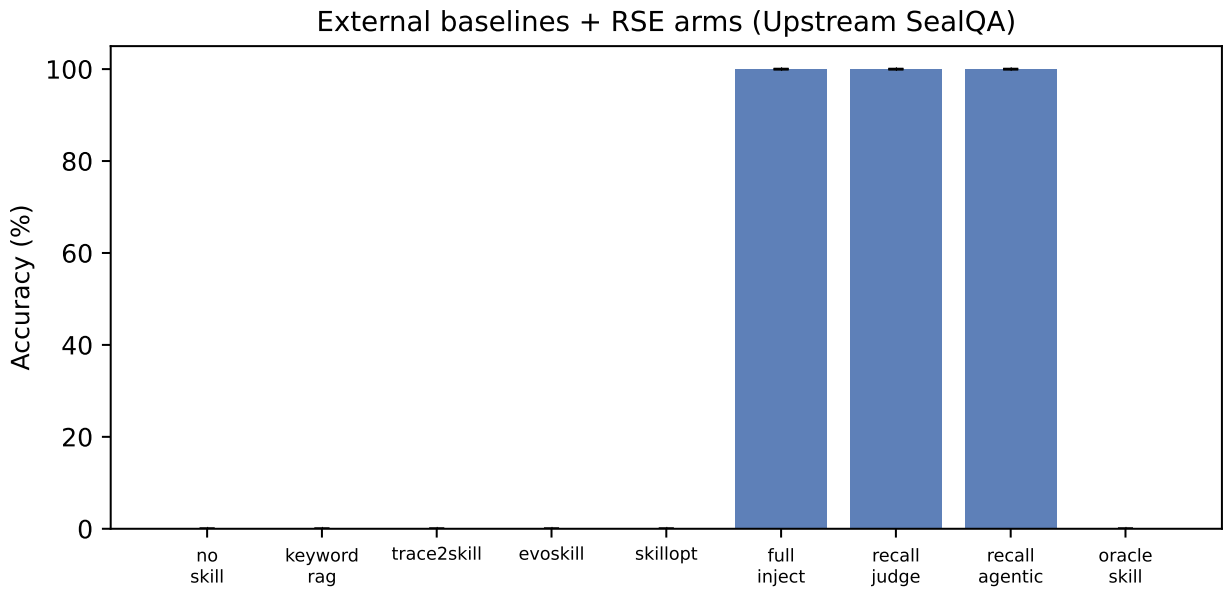


Figure 4: External skill-evolution baselines + RSE arms on upstream SealQA (Codex-graded). Error bars: bootstrap 95% CI.

Table 4: Cross-model transfer on WikiSkill probe (Codex inference, 3 samples).

Benchmark	full-inject	recall-agentic
ALFWorld	50%	**100%**
LiveMath	50%	50%
OfficeQA	100%	100%
SealQA	100%	100%
SpreadSheet	50%	50%
Overall	70%	**80%** (95% CI: 47–93%)
Mean tokens	534	**59**

(requires authenticated `claude` CLI). **Dogfood:** real-store measurements from one month of usage (N=29 nodes) reported in `EXPERIMENTS.md`.

5 Analysis and Limitations

RMC-Bench is hand-written (25 cases); cases from real captures would be more representative. **WikiSkill comparison** uses a 10-task probe plus full SealQA upstream split; HotPotQA/ALFWorld full splits not yet wired (HF import blocked). **External baselines** use inference-time proxies for Trace2Skill/EvoSkill/SkillOpt, not full evolution loops. **Multi-model:** Codex primary; Claude/Gemini cross-check recommended before claiming cross-model generalization. **Session study** is a single-turn proxy. **Dogfood** store is N=29; steady-state apex reduction unproven at scale. **Latency:** blocking recall $\sim$34s with accurate judge vs. $\sim$5s CLI floor.

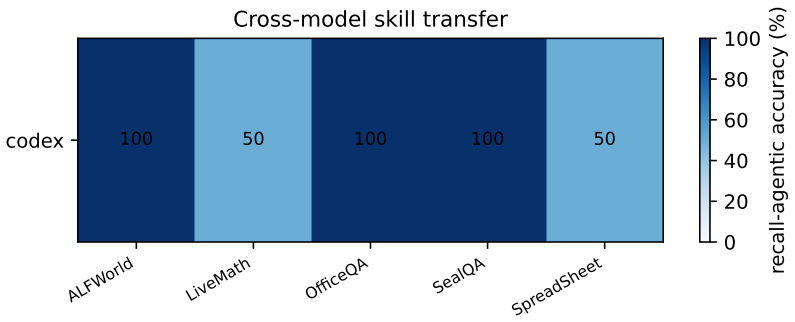


Figure 5: Cross-model transfer: recall-agentic accuracy by inference model (WikiSkill Table 2 style).

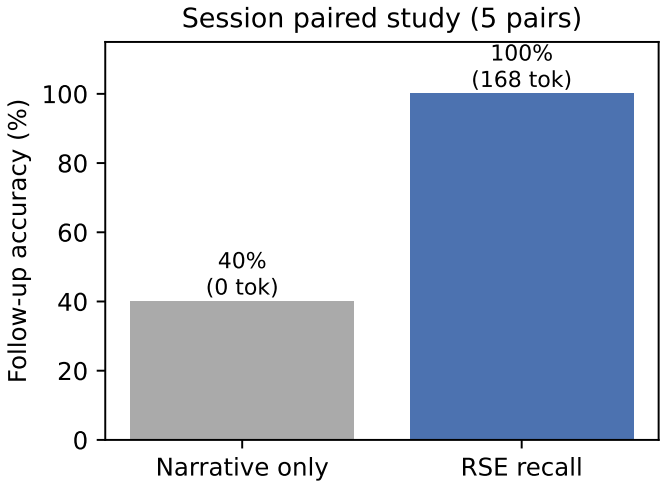


Figure 6: Session paired study: structured recall vs. narrative-only follow-up.

6 Related Work

Skill evolution. WikiSkill [Tang, 2026], Voyager [Wang et al., 2023], and Reflexion [Shinn et al., 2023] evolve procedural knowledge from traces but typically full-inject at test or keep retrieval separate from compaction validation.

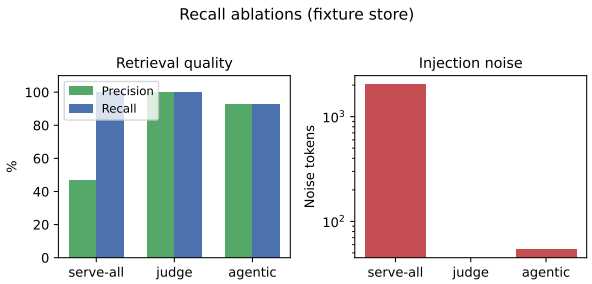
Skill retrieval and tools. RAG [Lewis et al., 2020], Toolformer [Schick et al., 2024], and ToolLLM [Qin et al., 2023] address retrieval and tool use without validated recursive compression of procedural memory.

Agent memory. MemGPT [Packer et al., 2023], generative agents [Park et al., 2023], and cognitive architectures for language agents [Sumers et al., 2024] manage long context but do not meta-test compressions against episode replay.

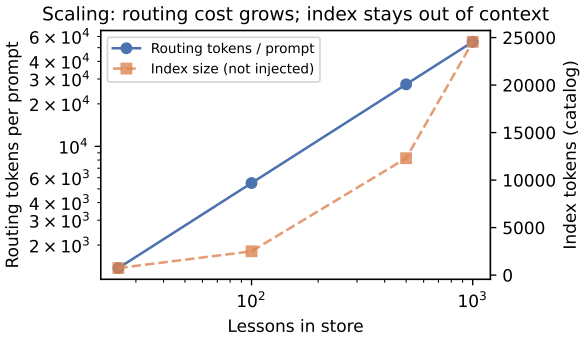
RSE combines WikiSkill's persistent-knowledge insight with explicit retrieval co-evolution and meta-tested compaction.

7 Conclusion

RSE compiles agent experience into recursively compressible lessons while co-evolving the retrieval that finds them. Meta-testing prevents silent behavioral drift; dynamic context discovery keeps catalog cost at zero injected tokens. RMC-Bench and scaling studies provide reproducible quantitative hooks; WikiSkill-



(a) Recall ablations



(b) Scaling study

Figure 7: Retrieval filtering removes noise; routing cost scales with store size while index stays out of context.

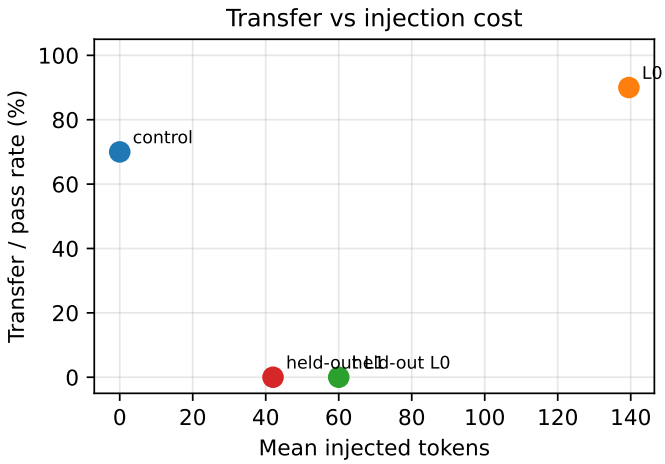


Figure 8: Transfer vs. injection cost on RMC-Bench core cases and held-out retention probe.

comparable evals show agentic recall beating full skill injection on accuracy (80% vs. 70%) at ∼88% lower token cost on a five-benchmark subset.

Reproducibility Statement

All evaluation harnesses, task YAML files, and result JSON artifacts ship with the repository. Run `scripts/validate_agent_harness.py --agent codex` then `scripts/run_all_experiments.py --agent codex --samples 3`. Figures regenerate via `scripts/generate_paper_figures.py`. See Appendix A for the NeurIPS-style checklist.

References

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Qinkai Yang, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.

et al. Tang. Wikiskill: Compiling agent experience into persistent knowledge for skill evolution. *arXiv preprint arXiv:2608.27454*, 2026.

et al. Vu. SealQA: A benchmark for web-augmented question answering with fresh evidence. *arXiv preprint*, 2024. HuggingFace: vtllms/sealqa.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Zhu, Yuke Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

A Reproducibility Checklist

This appendix follows the NeurIPS reproducibility checklist structure.

A.1 Scope of reproducibility

Claims supported by automated harness runs in this repository: (1) RMC-Bench transfer and retrieval scores; (2) recall ablation precision/recall/noise; (3) WikiSkill-comparable four-arm comparison; (4) session paired study lift; (5) synthetic scaling table. Claims from dogfood (`EXPERIMENTS.md`) are directional measurements on one user's store.

A.2 Algorithm

The full RSE loop is implemented in the `rmc` Python package: `recall`, `reflect`, `compact`, `tune`, and `eval-recall` subcommands. Design specification: `DESIGN.md`.

A.3 Dataset and benchmarks

- **RMC-Bench:** `evals/rmc-bench.yaml` (25 hand-written cases).
- **WikiSkill-comparable:** `evals/wikiskill-bench.yaml` (10 tasks, 5 domains).
- **Upstream SealQA:** `evals/upstream/sealqa-test.jsonl` (111 tasks, HF import).
- **MemGPT proxy:** `evals/memgpt-nested-kv.yaml` (8 nested-KV cases).
- **Session pairs:** `evals/session-pairs.yaml` (5 follow-up pairs).
- **Recall ablations:** fixture store under `evals/fixtures/recall-store/`.

A.4 Hyperparameters

- Grading agent: Codex (`gpt-5.6-sol`).
- Samples per case: 3 (majority vote).
- Compression ratio target $\rho = 0.7$.
- Replay pass threshold: 100%.

A.5 Computational resources

Full Codex suite (`run_all_experiments.py --agent codex --samples 3`) requires authenticated Codex CLI and approximately 30–60 minutes wall time depending on API latency. Preflight validation (`validate_agent_harness.py`) uses $\sim$4 Codex calls.

A.6 Code and data availability

- Repository: https://github.com/yiheinchai/rmc
- Results: `papers/rse/results/submission-latest.json`
- Figures: `papers/rse/figures/`

A.7 Experimental setup

```
pip install -e ".[dev]"
python3 scripts/validate_agent_harness.py --agent codex
python3 scripts/run_all_experiments.py --agent codex --samples 3
python3 scripts/generate_submission_report.py
python3 scripts/generate_paper_figures.py
cd papers/rse && make
```

A.8 Statistical significance

WikiSkill and upstream runs report bootstrap 95% confidence intervals and paired bootstrap tests vs. full-inject (`rmc/stats.py`). Default runs use 3 samples; increase with `--samples N` for tighter CIs.

A.9 Known limitations of reproduction

- Codex CLI authentication required; results vary with model version.
- Claude cross-check (`scripts/run_claude_crosscheck.sh`) requires separate `claude` authentication.
- WikiSkill subset is curated; full upstream splits not yet wired.

B Supplementary Tables

B.1 Dogfood retrieval (real store, N=29 nodes)

From `EXPERIMENTS.md` (Aug 2026):

Configuration	Precision	Recall	Noise tokens
Serve everything	28%	100%	15,917
Judge filter	48%	100%	7,146
Haiku router	35%	75%	8,818
After tune (1 round)	51%	88%	—

B.2 Scaling study (full table)

Lessons	Apexes	Index tok	Routing tok/prompt	Judge prec	Judge rec
25	25	740	1,375	67%	100%
100	100	2,503	5,500	67%	100%
500	500	12,303	27,500	67%	100%
1000	1000	24,553	55,000	67%	100%